

Java Backend Architecture Interview Series

Sub-Chapter 1.3

Collections -- Set Implementations

HashSet | LinkedHashSet | TreeSet | EnumSet | Set Algebra | ConcurrentSkipListSet |
equals/hashCode

40+ pages & 28 Q&A | Standard + Advanced Interview Questions | Working Code Examples

1 Set Interface Contract

1.1 Introduction & Motivation

A Set is a Collection that contains no duplicate elements. More formally, a Set contains no pair of elements e_1 and e_2 such that $e_1.equals(e_2)$, and at most one null element. The Set interface models the mathematical set abstraction. Sets are the correct tool whenever you need uniqueness enforcement, membership testing, deduplication, or set-theoretic operations (union, intersection, difference). Choosing the right Set implementation determines whether your operations are $O(1)$, $O(\log n)$, or constant time with a bit vector.

1.2 Set Interface Key Methods

```
import java.util.Set;
import java.util.stream.Collectors;

// java.util.Set<E> extends Collection<E>

boolean add(E e); // adds element; returns false if duplicate
boolean remove(Object o); // removes element; returns true if present
boolean contains(Object o); // returns true if element is present
int size(); // number of elements
boolean isEmpty();

Iterator<E> iterator(); // order depends on implementation
void forEach(Consumer<? super E> action); // Java 8+

// Bulk operations (implement Set algebra)
boolean addAll(Collection<? extends E> c); // union
boolean removeAll(Collection<?> c); // difference
boolean retainAll(Collection<?> c); // intersection
boolean containsAll(Collection<?> c); // subset check
void clear();

// Java 8+ default methods
set.removeIf(e -> e.startsWith("x")); // conditional remove
set.stream().filter(...).collect(Collectors.toSet());

// Java 9+ factory
Set<String> immutable = Set.of("a","b","c"); // immutable, null-hostile
```

Set interface key methods and factory

1.3 Set vs List: Key Differences

Feature	Set	List
Duplicates	Not allowed (enforced by equals())	Allowed

Feature	Set	List
Index access	No get(int index)	Yes -- get(int index)
Ordering	Depends on implementation	Insertion order preserved
Null elements	At most one (most impls); none in TreeSet/EnumSet	Multiple nulls allowed
Primary purpose	Membership testing, uniqueness	Ordered sequence
add() contract	Returns false if duplicate (no-op)	Always returns true

2 HashSet Internals O(1) Operations

2.1 Introduction The Workhorse Set

HashSet is the default Set implementation, offering O(1) average-case add(), remove(), and contains(). It is backed by a HashMap where elements are stored as keys mapping to a constant dummy value (PRESENT). This means HashSet inherits all of HashMap's internal mechanics: hash computation, bucket indexing, and collision handling via linked lists/trees. Understanding HashSet means understanding HashMap internals.

2.2 Before The Problem

```
import java.util.List;
import java.util.ArrayList;

// Without HashSet: manual deduplication is O(n^2)
List<String> list = List.of("a", "b", "a", "c", "b");
List<String> unique = new ArrayList<>();

for (String s : list) {
    if (!unique.contains(s)) unique.add(s); // O(n) per element -> O(n^2)
}
```

Before: O(n^2) deduplication without HashSet

2.3 After HashSet Solution

```
import java.util.Set;
import java.util.HashSet;
import java.util.List;

// HashSet: O(n) deduplication
List<String> list = List.of("a", "b", "a", "c", "b");
Set<String> unique = new HashSet<>(list); // constructor: O(n)
// unique = {a, b, c} order is NOT guaranteed!

// Core operations -- all O(1) average case
Set<String> set = new HashSet<>();
set.add("hello"); // true (added)
set.add("hello"); // false (duplicate -- no-op)
set.contains("hello"); // true O(1)
set.remove("hello"); // true O(1)
set.size(); // 0

// Null IS allowed in HashSet (one null maximum)
set.add(null); // OK
```

```
set.add(null); // false (already present)

set.contains(null); // true
```

HashSet: deduplication and core operations

2.4 HashSet Internals: HashMap Backing

```
// HashSet source code (simplified):

public class HashSet<E> {

    private HashMap<E, Object> map;

    private static final Object PRESENT = new Object();

    public boolean add(E e) {

        return map.put(e, PRESENT) == null; // true if no previous mapping
    }

    public boolean contains(Object o) { return map.containsKey(o); }

    public boolean remove(Object o) { return map.remove(o) == PRESENT; }

    public Iterator<E> iterator() { return map.keySet().iterator(); }

}

// Bucket mechanics (identical to HashMap):

// 1. Compute hash: h = key.hashCode() ^ (h >>> 16) (spread)

// 2. Find bucket: index = h & (capacity - 1)

// 3. Collision: linked list -> tree at TREEIFY_THRESHOLD (8)

// 4. Load factor: default 0.75; resize when size > capacity * 0.75

// 5. Resize: double capacity; O(n) rehash of all elements
```

HashSet internal mechanics: HashMap backing and bucket structure

2.5 Impact of hashCode on HashSet Performance

WARNING: If your element class returns a constant hashCode() (e.g., always 0), ALL elements land in the same bucket. HashSet degrades from $O(1)$ to $O(n)$ -- or $O(\log n)$ after treeification. ALWAYS implement a well-distributed hashCode when using custom objects in HashSet.

```
import java.util.Objects;

// BAD: all elements hash to bucket 0 -> O(n) ops

class BadKey {

    int val;

    public int hashCode() { return 0; } // constant hash!

    public boolean equals(Object o) { ... }
```

```
}

// GOOD: well-distributed hash
class GoodKey {
    int val;

    public int hashCode() { return Objects.hash(val); } // distributed
    public boolean equals(Object o) {
        if (!(o instanceof GoodKey k)) return false;
        return val == k.val;
    }
}
```

hashCode quality: bad vs good distribution

3 LinkedHashMap Insertion-Order Set

3.1 Introduction Hash Performance + Predictable Order

LinkedHashSet extends HashSet and adds a doubly-linked list running through all entries. This linked list defines the iteration order: the order in which elements were inserted. It provides $O(1)$ add, remove, and contains (same as HashSet) but with predictable iteration order. The memory overhead is approximately one extra doubly-linked list node per element. Use LinkedHashSet when you need both uniqueness and insertion order.

3.2 LinkedHashMap vs HashSet

```
import java.util.Set;
import java.util.HashSet;
import java.util.LinkedHashSet;
import java.util.List;

// HashSet: iteration order NOT guaranteed
Set<String> hash = new HashSet<>(List.of("banana","apple","cherry","apricot"));
System.out.println(hash); // unpredictable: [apple, cherry, banana, apricot] etc.

// LinkedHashSet: insertion order guaranteed
Set<String> linked = new LinkedHashSet<>(List.of("banana","apple","cherry","apricot"));
System.out.println(linked); // [banana, apple, cherry, apricot] insertion order

// Same API as HashSet; same  $O(1)$  complexities
linked.add("date"); //  $O(1)$  -- adds to hash + end of linked list
linked.contains("apple"); //  $O(1)$ 
linked.remove("banana"); //  $O(1)$  -- removes from hash + unlinks from chain
System.out.println(linked); // [apple, cherry, apricot, date]
```

LinkedHashSet: insertion-order iteration vs HashSet

3.3 Deduplication with Order Preservation

TIP: LinkedHashSet is the go-to tool for order-preserving deduplication of a List. Convert the List to LinkedHashSet to remove duplicates, then back to List to preserve order.

```
import java.util.Set;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.ArrayList;
import java.util.stream.Collectors;

// Use case: deduplicate a list while preserving insertion order
List<String> names = List.of("Alice","Bob","Alice","Carol","Bob");
```

```
// HashSet: unordered (do NOT use when order matters)

Set<String> unordered = new HashSet<>(names);

// LinkedHashSet: ordered deduplication

List<String> deduped = new ArrayList<>(new LinkedHashSet<>(names));

System.out.println(deduped); // [Alice, Bob, Carol] order preserved!

// Stream approach: also preserves encounter order for ordered sources

List<String> deduped2 = names.stream().distinct().collect(Collectors.toList());

// distinct() uses a HashSet internally for tracking; order is source order
```

Order-preserving deduplication with LinkedHashSet

4 TreeSet Sorted NavigableSet

4.1 Introduction Sorted Set with Range Operations

TreeSet implements the NavigableSet and SortedSet interfaces, backed by a Red-Black tree (same as TreeMap). Elements are stored in sorted order according to either their natural ordering (Comparable) or a Comparator provided at construction. All operations are $O(\log n)$. TreeSet is the right choice when you need sorted iteration or range queries -- features that HashSet simply does not provide.

4.2 Before The Problem with Unsorted Sets

```
import java.util.Set;
import java.util.HashSet;
import java.util.List;
import java.util.stream.Collectors;

// HashSet: cannot find min, max, range efficiently
Set<Integer> scores = new HashSet<>(List.of(85,92,78,95,60));

// To find max: must iterate all  $O(n)$ 
int max = scores.stream().mapToInt(Integer::intValue).max().getAsInt();

// To find scores between 80 and 90: must iterate and filter  $O(n)$ 
Set<Integer> range = scores.stream()
    .filter(s -> s >= 80 && s <= 90)
    .collect(Collectors.toSet());
```

Before: range operations on HashSet require $O(n)$ scans

4.3 After TreeSet Range Navigation

```
import java.util.TreeSet;
import java.util.List;
import java.util.SortedSet;
import java.util.NavigableSet;

// TreeSet:  $O(\log n)$  range operations
TreeSet<Integer> scores = new TreeSet<>(List.of(85,92,78,95,60));
System.out.println(scores); // [60, 78, 85, 92, 95] sorted!

// NavigableSet methods: all  $O(\log n)$ 
scores.first(); // 60 smallest
scores.last(); // 95 largest
scores.lower(85); // 78 strictly less than 85
scores.floor(85); // 85 less than or equal to 85
scores.higher(85); // 92 strictly greater than 85
```

```
scores.ceiling(80); // 85 greater than or equal to 80

// Range views (backed by original set, live views)
SortedSet<Integer> inRange = scores.subSet(80, true, 92, true); // [85,92]
SortedSet<Integer> headSet = scores.headSet(85); // [60,78]
SortedSet<Integer> tailSet = scores.tailSet(85); // [85,92,95]

// Reverse order
NavigableSet<Integer> desc = scores.descendingSet(); // [95,92,85,78,60]
Integer pollMax = scores.pollLast(); // removes and returns 95
```

TreeSet: sorted order and NavigableSet range operations

4.4 TreeSet with Custom Comparator

```
import java.util.TreeSet;
import java.util.Comparator;
import java.util.List;

// Custom ordering: sort strings by length, then alphabetically
TreeSet<String> byLength = new TreeSet<>(
    Comparator.comparingInt(String::length).thenComparing(Comparator.naturalOrder())
);
byLength.addAll(List.of("banana", "fig", "apple", "kiwi", "date"));
System.out.println(byLength); // [fig, date, kiwi, apple, banana]

// WARNING: TreeSet uses comparator/compareTo for equality, NOT equals()!
// If comparator returns 0 for two objects, they are treated as DUPLICATES
// even if equals() returns false!
TreeSet<String> caseInsensitive = new TreeSet<>(String.CASE_INSENSITIVE_ORDER);
caseInsensitive.add("Hello");
caseInsensitive.add("hello"); // duplicate! CASE_INSENSITIVE_ORDER returns 0
System.out.println(caseInsensitive.size()); // 1 only "Hello" stored
```

TreeSet with custom Comparator and the comparator-equals trap

WARNING: `TreeSet` uses `compareTo()` or the `Comparator` for equality checks, NOT `equals()`. If the `Comparator` returns 0 for two distinct objects (by `equals()`), `TreeSet` treats them as duplicates and discards the second. The `Comparator` must be consistent with `equals()` or this causes surprises.

4.5 TreeSet vs HashSet vs LinkedHashSet

Feature	HashSet	LinkedHashSet	TreeSet
Ordering	None (unpredictable)	Insertion order	Sorted (natural/Comparator)
add/remove/contains	O(1) avg	O(1) avg	O(log n)

Feature	HashSet	LinkedHashSet	TreeSet
Null allowed?	Yes (one null)	Yes (one null)	NO -- NullPointerException
Memory overhead	Baseline	+linked list node	+Red-Black tree node (~3 refs)
Range queries	No	No	Yes -- subSet/headSet/tailSet
Use when	Uniqueness + O(1) lookup	Uniqueness + order	Sorted iteration + range ops

5 EnumSet Bit Vector Set for Enums

5.1 Introduction The Most Efficient Set for Enum Types

EnumSet is a specialised Set implementation designed exclusively for Enum types. Internally it uses a bit vector (a long or long[] for large enums): each enum constant corresponds to one bit position determined by its ordinal value. This gives extremely fast O(1) operations with minimal memory footprint. For small enums (up to 64 constants) a single 64-bit long is sufficient. EnumSet is always the correct choice when working with sets of enum constants -- it is faster and more memory-efficient than HashSet by a large margin.

5.2 EnumSet Factory Methods

```
import java.util.EnumSet;

enum Day { MON, TUE, WED, THU, FRI, SAT, SUN }

// Factory methods no public constructor

EnumSet<Day> allDays = EnumSet.allOf(Day.class); // all 7 days
EnumSet<Day> noDays = EnumSet.noneOf(Day.class); // empty set
EnumSet<Day> weekend = EnumSet.of(Day.SAT, Day.SUN); // 2 elements
EnumSet<Day> weekdays = EnumSet.complementOf(weekend); // MON-FRI
EnumSet<Day> midWeek = EnumSet.range(Day.WED, Day.FRI); // WED, THU, FRI

// Standard Set operations all O(1) bit operations

weekend.add(Day.FRI); // add FRI to weekend
weekend.contains(Day.SUN); // true O(1) bit test
weekend.remove(Day.FRI); // remove O(1) bit clear

// Null rejected

// weekend.add(null); // NullPointerException

// Iteration: always in ordinal order (MON before TUE before WED etc.)
for (Day d : weekend) System.out.println(d); // SAT, SUN
```

EnumSet factory methods and operations

5.3 EnumSet Internal Bit Vector Implementation

```
// For enums with <= 64 constants: RegularEnumSet uses a single long
// For enums with > 64 constants: JumboEnumSet uses long[]

// Bit representation for Day enum (ordinals: MON=0, TUE=1, ... SUN=6):
// EnumSet.of(Day.SAT, Day.SUN):
// bit 5 (SAT) = 1, bit 6 (SUN) = 1
```

```
// binary: 0b1100000 = 96 (as a long)

// add(e) : elements |= (1L << e.ordinal())
// remove(e): elements &= ~(1L << e.ordinal())
// contains(e): (elements & (1L << e.ordinal())) != 0

// EnumSet vs HashSet<Day> performance (approximate)
// EnumSet.contains(): 1 bit AND operation
// HashSet.contains(): hashCode() + array index + equals()
// EnumSet is typically 10-20x faster for enum operations
```

EnumSet bit vector internals: add/remove/contains as bit operations

5.4 Permission/Flag Pattern with EnumSet

```
import java.util.EnumSet;

enum Permission { READ, WRITE, EXECUTE, DELETE }

// Instead of int bitmask (error-prone, not type-safe)
// int perms = READ_BIT | WRITE_BIT; // easy to mix up constants

// Use EnumSet type-safe, readable, efficient
EnumSet<Permission> userPerms = EnumSet.of(Permission.READ, Permission.WRITE);
EnumSet<Permission> adminPerms = EnumSet.allOf(Permission.class);

boolean canDelete = userPerms.contains(Permission.DELETE); // false
boolean hasAll = adminPerms.containsAll(userPerms); // true

// Grant permission
userPerms.add(Permission.EXECUTE);

// Revoke permission
userPerms.remove(Permission.WRITE);

// Clone for immutable view
EnumSet<Permission> snapshot = userPerms.clone();
```

Permission/flag pattern using EnumSet instead of int bitmasks

TIP: `EnumSet.copyOf(collection)` requires the collection to be non-empty (throws `IllegalArgumentException` on empty). For empty `EnumSet`: use `EnumSet.noneOf(MyEnum.class)`. Always prefer `EnumSet` over `HashSet<SomeEnum>` -- it is faster, more memory-efficient, and self-documenting.

6 Set Algebra: Union, Intersection & Difference

6.1 Introduction Mathematical Set Operations in Java

The Set interface provides bulk operation methods that directly implement set-theoretic operations. These are essential for computing unique combinations, overlaps, and exclusive elements across collections. Understanding these methods enables elegant solutions to deduplication, permission comparison, and data analysis problems.

6.2 Three Set Operations with Code

```
import java.util.Set;
import java.util.HashSet;
import java.util.Collections;

Set<Integer> setA = new HashSet<>(Set.of(1,2,3,4,5));
Set<Integer> setB = new HashSet<>(Set.of(3,4,5,6,7));

// UNION: elements in A OR B (addAll)
Set<Integer> union = new HashSet<>(setA);
union.addAll(setB);
System.out.println(union); // [1,2,3,4,5,6,7]

// INTERSECTION: elements in A AND B (retainAll)
Set<Integer> intersection = new HashSet<>(setA);
intersection.retainAll(setB);
System.out.println(intersection); // [3,4,5]

// DIFFERENCE A minus B: elements in A but NOT in B (removeAll)
Set<Integer> difference = new HashSet<>(setA);
difference.removeAll(setB);
System.out.println(difference); // [1,2]

// SYMMETRIC DIFFERENCE: elements in A OR B but NOT BOTH
Set<Integer> symDiff = new HashSet<>(union);
symDiff.removeAll(intersection);
System.out.println(symDiff); // [1,2,6,7]

// SUBSET CHECK: is setA a subset of setB?
boolean isSubset = setB.containsAll(setA); // false

// DISJOINT CHECK: do A and B share NO elements?
boolean disjoint = Collections.disjoint(setA, setB); // false
```

Set algebra: union, intersection, difference, symmetric difference

WARNING: `addAll()`, `retainAll()`, and `removeAll()` MUTATE the receiver set. Always work on a COPY (new `HashSet<>(setA)`) if you need to preserve the original set.

6.3 Stream-Based Set Operations

```
import java.util.Set;
import java.util.stream.Stream;
import java.util.stream.Collectors;

Set<String> devs = Set.of("Alice", "Bob", "Carol");
Set<String> managers = Set.of("Bob", "Dave");

// Union via Stream.concat
Set<String> union = Stream.concat(devs.stream(), managers.stream())
    .collect(Collectors.toSet());

// Intersection via filter
Set<String> both = devs.stream()
    .filter(managers::contains)
    .collect(Collectors.toSet()); // {Bob}

// Difference: devs who are NOT managers
Set<String> onlyDevs = devs.stream()
    .filter(d -> !managers.contains(d))
    .collect(Collectors.toSet()); // {Alice, Carol}
```

Set algebra using Stream API

7 equals/hashCode Requirement for Sets

7.1 The Problem Custom Objects That "Vanish" from Sets

HashSet uses hashCode() to find the bucket and equals() to confirm identity within the bucket. If a class overrides equals() but not hashCode(), equal objects may hash to DIFFERENT buckets, causing the Set to contain apparent duplicates and contains() to return false for elements you know are in the Set. This section shows the exact failure modes and the correct fix.

7.2 Broken Example: equals but No hashCode

```
class User {  
  
    String name;  
  
    User(String name) { this.name = name; }  
  
    @Override  
    public boolean equals(Object o) {  
        if (!(o instanceof User u)) return false;  
        return name.equals(u.name);  
    }  
  
    // hashCode() NOT overridden -- uses Object.hashCode() = identity hash!  
}  
  
Set<User> users = new HashSet<>();  
  
User alice1 = new User("Alice");  
User alice2 = new User("Alice");  
  
users.add(alice1);  
  
System.out.println(users.contains(alice2)); // false! (different buckets)  
  
users.add(alice2); // added! (different bucket)  
  
System.out.println(users.size()); // 2 should be 1 BUG!
```

Broken: equals without hashCode causes set duplicates

7.3 Fixed: equals and hashCode Together

```
import java.util.Objects;  
import java.util.Set;  
import java.util.HashSet;
```

```
class User {  
    String name;  
    User(String name) { this.name = name; }  
  
    @Override  
    public boolean equals(Object o) {  
        if (!(o instanceof User u)) return false;  
        return Objects.equals(name, u.name);  
    }  
  
    @Override  
    public int hashCode() {  
        return Objects.hash(name); // SAME field as equals!  
    }  
}  
  
Set<User> users = new HashSet<>();  
users.add(new User("Alice"));  
users.add(new User("Alice")); // duplicate detected!  
System.out.println(users.size()); // 1 correct  
System.out.println(users.contains(new User("Alice"))); // true correct  
  
// Best alternative: Java record generates correct equals/hashCode  
record UserRecord(String name) {}  
Set<UserRecord> recs = new HashSet<>();  
recs.add(new UserRecord("Alice"));  
recs.add(new UserRecord("Alice")); // duplicate -- size stays 1
```

Fixed: hashCode consistent with equals

8 ConcurrentSkipListSet & Thread-Safe Sets

8.1 Introduction Concurrent Set Implementations

In multi-threaded environments, neither `HashSet`, `LinkedHashSet`, nor `TreeSet` is thread-safe. Java provides two thread-safe `Set` implementations: `CopyOnWriteArraySet` (backed by `CopyOnWriteArrayList` -- suitable for tiny read-heavy sets) and `ConcurrentSkipListSet` (backed by `ConcurrentSkipListMap` -- sorted, lock-free concurrent operations). Additionally, `Collections.synchronizedSet()` wraps any `Set` with a synchronized mutex.

8.2 ConcurrentSkipListSet Sorted + Thread-Safe

```
import java.util.concurrent.ConcurrentSkipListSet;

// ConcurrentSkipListSet: sorted, thread-safe, O(log n) ops
// Backed by ConcurrentSkipListMap
// All NavigableSet operations are lock-free and thread-safe
ConcurrentSkipListSet<Integer> scores = new ConcurrentSkipListSet<>();

// Thread-safe add from multiple threads (no external sync needed)
scores.add(85);
scores.add(92);
scores.add(78);

// NavigableSet operations (all lock-free)
scores.first(); // 78 O(log n)
scores.last(); // 92
scores.ceiling(80); // 85
scores.pollFirst(); // removes and returns 78 atomic

// Iteration is weakly consistent:
// - Never throws ConcurrentModificationException
// - May or may not reflect concurrent add()/remove() calls
for (int s : scores) System.out.println(s); // safe, weakly consistent
```

ConcurrentSkipListSet: sorted, thread-safe, lock-free

8.3 Thread-Safe Set Options Comparison

Option	Ordering	Null ?	Thread Safety	When to Use
<code>Collections.synchronizedSet(hashSet)</code>	None	Yes	Synchronized mutex (one thread at a time)	Simple multi-thread needs; iterator must sync externally
<code>CopyOnWriteArraySet</code>	Insertion order	Yes	Reads lock-free; writes copy array	Tiny sets, read-heavy, rare writes

Option	Ordering	Null ?	Thread Safety	When to Use
ConcurrentSkipListSet	Sorted	No	Lock-free (CAS); concurrent read+write	Sorted set with concurrent updates

8.4 Creating a Thread-Safe HashSet (No Built-In)

```
import java.util.Set;
import java.util.HashSet;
import java.util.Collections;
import java.util.concurrent.ConcurrentHashMap;

// Option 1: Collections.synchronizedSet() synchronized wrapper
Set<String> syncSet = Collections.synchronizedSet(new HashSet<>());

syncSet.add("a"); // synchronized
syncSet.contains("a"); // synchronized

// ITERATION MUST BE MANUALLY SYNCHRONIZED:
synchronized (syncSet) {
    for (String s : syncSet) System.out.println(s);
}

// Option 2: ConcurrentHashMap.newKeySet() concurrent, better throughput
Set<String> concSet = ConcurrentHashMap.newKeySet(); // Java 8+
concSet.add("a"); // lock-free concurrent add
concSet.contains("a"); // lock-free

for (String s : concSet) System.out.println(s); // weakly consistent, safe

// Option 2 is preferred for general concurrent HashSet needs
```

Thread-safe HashSet options: synchronizedSet vs ConcurrentHashMap.newKeySet()

Q&A Interview Questions & Answers 28 Questions

Standard questions are mid-level. [ADV] questions target senior/principal interviews.

Part 1 Set Interface & Contract

Q1. What guarantees does the Set interface make?

No duplicate elements (no two elements e_1, e_2 where $e_1.equals(e_2)$). At most one null element (enforced by most implementations). `add()` returns false for duplicates (no exception thrown). No ordering guarantee beyond what the specific implementation provides.

Q2. How does Set enforce no duplicates?

It relies on `equals()` for equality checking. For `HashSet`, it uses `hashCode()` to find the bucket, then `equals()` to confirm. If `hashCode()` maps to the same bucket and `equals()` returns true, the element is considered a duplicate and `add()` returns false. For `TreeSet`, `compareTo()/Comparator` is used -- NOT `equals()`.

Q3. What is the difference between `add()` returning false vs throwing an exception?

`add(E e)` returns false (not an exception) when the element is already present -- adding a duplicate to a Set is a valid, expected operation. The return value tells you whether the set was changed. This is part of the `Collection.add()` contract: 'returns true if this collection changed as a result of the call'.

Part 2 HashSet

Q4. What data structure backs HashSet internally?

A `HashMap`, where the element is the key and a constant `PRESENT` object is the value. This means `HashSet` inherits all of `HashMap`'s internals: hash-based bucketing, chaining for collisions, treeification at threshold 8, and resizing at load factor 0.75.

Q5. Why is HashSet not ordered?

`HashSet` stores elements in buckets based on their `hashCode`. The bucket index is `hash & (capacity-1)`. Different `hashCode`s lead to different bucket indices, and iteration traverses buckets sequentially. Since `hashCode`s of typical objects are scattered, iteration order appears random and changes on resize.

Q6. What happens if all elements in a HashSet have the same hashCode?

All elements land in the same bucket, forming a collision chain. Performance degrades from $O(1)$ to $O(n)$ for `add/remove/contains`. With Java 8+ treeification: if a bucket has ≥ 8 elements, it converts to a Red-Black tree, recovering to $O(\log n)$. This is why poor `hashCode` implementations (returning 0 or a constant) are a significant performance hazard.

Q7. What is the default load factor of HashSet and what does it mean?

0.75 (inherited from `HashMap`). When the number of elements exceeds `capacity * 0.75`, `HashSet` doubles its capacity and rehashes all elements ($O(n)$). A lower load factor (e.g., 0.5) reduces collisions but wastes memory. A higher load factor (e.g., 1.0) saves memory but increases collisions. 0.75 is a good empirical balance.

Q8. Can HashSet contain null? What about the other Set implementations?

`HashSet`: YES -- one null allowed (stored in bucket 0, since `null.hashCode()` would NPE, so `HashSet/HashMap` special-case null keys). `LinkedHashSet`: YES -- same as `HashSet`. `TreeSet`: NO -- `compareTo()` is called on elements for ordering; `null.compareTo()` throws NPE. `EnumSet`: NO -- `NullPointerException` immediately. `ConcurrentSkipListSet`: NO -- NPE.

Part 3 LinkedHashSet & TreeSet

Q9. How does LinkedHashSet maintain insertion order?

`LinkedHashSet` extends `HashSet` and adds a doubly-linked list threading through all entries. Each node in the backing `LinkedHashMap` has `before/after` pointers. When iterating, it follows the linked list order (insertion order) rather than bucket order. The extra overhead: one doubly-linked list node per element (approximately 12 bytes extra).

Q10. When should you use LinkedHashSet over HashSet?

When you need both uniqueness AND predictable iteration order matching insertion order. Common use cases: deduplication while preserving order of first appearance, building a history of unique events, or any API where the caller expects consistent

iteration order. The performance cost is negligible ($O(1)$ same as `HashSet`) with slightly more memory.

Q11. What is the time complexity of `TreeSet` operations?

$O(\log n)$ for add, remove, contains -- all tree traversals. $O(\log n)$ for `NavigableSet` operations: `first()`, `last()`, `lower()`, `floor()`, `higher()`, `ceiling()`. $O(n)$ for iteration. $O(\log n)$ for `subSet/headSet/tailSet` creation (view, not copy). Compare to `HashSet` $O(1)$ -- `TreeSet` is slower but provides sorted iteration and range queries.

Q12. Why does `TreeSet` not allow null elements?

`TreeSet` uses `compareTo()` or `Comparator` to order elements. When adding null, `TreeSet` calls `compareTo(null)` on the natural ordering, which throws `NullPointerException` (`Comparable` implementations are not expected to handle null). Even if you provide a `Comparator` that handles null, most implementations still reject it. If you need null in a sorted context, consider using `Optional` or a sentinel value.

Q13. What is the consistency requirement for `TreeSet` Comparators?

The `Comparator` must be consistent with `equals()`: if `comparator.compare(a,b)==0` then `a.equals(b)` should be true. If not, the Set may appear to contain elements that are 'equal by comparator' but different by `equals()`. The Javadoc calls this behaviour 'inconsistent with equals' and notes it can cause problems in some `SortedSet` implementations.

Part 4 EnumSet

Q14. What is `EnumSet` and why is it faster than `HashSet`?

`EnumSet` is a Set implementation for enum types backed by a bit vector (a long or long[]). Each enum constant occupies one bit at its ordinal position. `add()` = bit OR, `remove()` = bit AND-NOT, `contains()` = bit AND -- all single-instruction CPU operations. `HashSet` requires `hashCode()` computation, array indexing, and `equals()` calls. `EnumSet` is typically 10-20x faster for enum operations.

Q15. What are the two internal `EnumSet` implementations?

`RegularEnumSet`: for enums with ≤ 64 constants. Uses a single long as the bit vector. All operations are truly $O(1)$: one bit manipulation on a long. `JumboEnumSet`: for enums with > 64 constants. Uses long[] with `floor(ordinal/64)` as array index. The JVM chooses the implementation automatically via `EnumSet.of()` / `EnumSet.allOf()` factory methods.

Q16. What does `EnumSet.complementOf()` do?

Returns an `EnumSet` containing all elements of the same enum type NOT present in the given `EnumSet`. `EnumSet weekend = EnumSet.of(Day.SAT, Day.SUN)`; `EnumSet weekdays = EnumSet.complementOf(weekend)`; // MON,TUE,WED,THU,FRI. Internally: bitwise NOT of the elements field, masked to only valid ordinal bits.

Part 5 Set Algebra

Q17. How do you compute the intersection of two Sets in Java?

Set intersection = `new HashSet<>(setA); intersection.retainAll(setB)`; `retainAll` removes all elements from intersection that are NOT in setB. This is $O(\min(|A|, |B|))$ -- `retainAll` iterates one set and calls `contains()` on the other. To avoid mutating setA: always work on a copy.

Q18. What is the time complexity of `Set.containsAll()`?

$O(n)$ where n = size of the argument collection. It iterates over the argument and calls `this.contains()` on each element. For `HashSet`: each `contains()` is $O(1)$, so `containsAll` is $O(n)$ total. For `TreeSet`: each `contains()` is $O(\log m)$ where m = `TreeSet` size, so `containsAll` is $O(n \log m)$.

Part 6 Advanced Questions [ADV]

Q19. [ADV] How does `HashSet` handle hash collisions internally in Java 8+?

Java 8+: collision bucket starts as a linked list. When a bucket size $\geq$ `TREEIFY_THRESHOLD` (8) AND total capacity $\geq$ `MIN_TREEIFY_CAPACITY` (64), the linked list is converted to a Red-Black tree. This degrades $O(n)$ worst case to $O(\log n)$. When elements are removed and bucket size falls to $\leq$ `UNTREEIFY_THRESHOLD` (6), the tree is converted back to a linked list.

Q20. [ADV] What is the difference between `ConcurrentHashMap.newKeySet()` and `synchronizedSet(new HashSet<>())`?

`synchronizedSet()`: wraps with a single mutex -- all operations serialize. Iteration requires external synchronization. Under high concurrency, threads queue on the one lock. `ConcurrentHashMap.newKeySet()`: internally segments the map with fine-grained locks (16 segments by default). Concurrent reads are lock-free. Multiple writers can proceed in different

segments. Iteration is weakly consistent (no CME). Much higher throughput under contention.

Q21. [ADV] Explain the skip list data structure used in ConcurrentSkipListSet.

A skip list is a probabilistic data structure with multiple layers of linked lists. The bottom layer contains all elements in sorted order. Higher layers are 'express lanes' containing a subset of elements (each element appears in the next layer with probability 0.5). Search: start at top layer, skip forward until overshoot, drop down a layer, repeat. Expected $O(\log n)$ for search/insert/delete. Lock-free via CAS operations on node pointers.

Q22. [ADV] How does TreeSet efficiently implement subSet() range views?

`TreeSet.subSet(from, fromInclusive, to, toInclusive)` returns a descending view of the underlying `TreeMap`'s `subMap()`. This is a LIVE VIEW backed by the same Red-Black tree with from/to bounds. No copying occurs: add/remove on the subSet view modifies the backing `TreeSet` (within bounds). Operations outside the range throw `IllegalArgumentException`. Complexity: creating the view $O(1)$; operations within the view $O(\log n)$ with bounds checking.

Q23. [ADV] What is the memory layout difference between EnumSet and HashSet?

`EnumSet` with up to 64 constants: 1 long (8 bytes) for the bit vector + object header (~16 bytes) = ~24 bytes total for ANY size. `HashSet`: `HashMap` object (~48 bytes) + `Entry[]` array (~16 + 4*capacity bytes) + per-entry `Node` (~32 bytes). For 10 enum constants in a full `EnumSet`: 24 bytes. In `HashSet`: ~400+ bytes. `EnumSet` is dramatically more compact for enum sets.

Q24. [ADV] How does Set.copyOf() differ from new HashSet<>(set)?

`Set.copyOf(set)` (Java 10+): returns an IMMUTABLE copy, null-hostile, and may use a compact internal representation. It is not a `HashSet` -- it is an unmodifiable set with unknown implementation. `new HashSet<>(set)`: returns a MUTABLE copy backed by a `HashMap`. Allows add/remove. Allows null. `Set.copyOf` is preferred when you need a guaranteed-immutable defensive copy for APIs.

Q25. [ADV] What happens when you use a mutable object as a HashSet element and then mutate it?

The element's `hashCode()` changes after mutation, but it was placed in a bucket based on the old `hashCode`. `contains()` now computes the new `hashCode`, looks in the wrong bucket, and returns false. The element is effectively lost: it cannot be found or removed. The `set.size()` is still correct but the element is 'ghost' data in the wrong bucket. Fix: use only immutable objects (or records) as `Set` elements, or remove before mutating and re-add after.

Q26. [ADV] How does LinkedHashSet implement iteration order with O(1) all ops?

`LinkedHashSet` extends `HashSet`, which is backed by a `LinkedHashMap`. `LinkedHashMap` extends `HashMap` and adds before/after pointers to each `Entry` node, forming a doubly-linked list through all entries in insertion order. `add()`: hash insert ($O(1)$) + append to tail of linked list ($O(1)$). `remove()`: hash remove ($O(1)$) + unlink from doubly-linked list ($O(1)$ with prev/next pointers). `iterator()`: follows the linked list from head to tail.

Part 7 Real-World Scenarios [ADV]

Q27. [ADV] A report shows HashSet.contains() is slow under high concurrency. What are the causes and fixes?

Causes: (1) `HashSet` is not thread-safe -- concurrent reads+writes cause data corruption, not just slowness. (2) Lock contention on `synchronizedSet` wrapper -- single mutex bottleneck. (3) Poor `hashCode()` causing $O(n)$ bucket scan. Fixes: (1) Replace with `ConcurrentHashMap.newKeySet()` for concurrent $O(1)$ contains. (2) Fix `hashCode()` implementation for better distribution. (3) Consider read-write lock pattern if reads vastly outnumber writes.

Q28. [ADV] Design a deduplication pipeline that processes 10M strings from a file keeping insertion order.

Solution: Use `LinkedHashSet` (preserves insertion order + $O(1)$ contains). Read line by line (avoid loading all into memory at once if possible). Pre-size: `new LinkedHashSet<>(capacity, 0.75f)` to avoid resize. If memory is a concern: use `BloomFilter` (Guava) for probabilistic pre-filter then exact check. For truly huge datasets that don't fit in RAM: sort the file externally, then stream with duplicate detection via comparison to previous element.

EX Exercises, Debug Challenges & MCQ

Part A Coding Challenges

- A1.** Implement `findDuplicates(List list)` that returns a `Set` of elements appearing more than once. Use $O(n)$ time and $O(n)$ space.
- A2.** Given two `List` representing search result pages, compute: (1) all unique results, (2) results on both pages, (3) results exclusive to page 1. Use `Set` operations.
- A3.** Implement a word frequency counter that identifies the `K` most frequent words from a file, using a combination of `HashMap` and `TreeSet/PriorityQueue`.
- A4.** Write a method that given a `Set` of role names and a required role, returns `true` if the set contains the required role, case-insensitively. Hint: use `TreeSet` with `CASE_INSENSITIVE_ORDER`.

Part B Debug & Fix

```
// B1 Missing hashCode causes phantom duplicates

class Point { int x, y;

public boolean equals(Object o) {

if (!(o instanceof Point p)) return false;

return x==p.x && y==p.y;

}

// hashCode() NOT overridden!

}

Set<Point> pts = new HashSet<>();

pts.add(new Point(1,2));

pts.add(new Point(1,2)); // should be duplicate -- but NOT detected!

System.out.println(pts.size()); // 2 BUG

// FIX: add public int hashCode() { return Objects.hash(x,y); }
```

B1: Fix missing hashCode causing phantom duplicates

```
import java.util.Set;
import java.util.HashSet;
import java.util.List;
import java.util.ArrayList;

// B2 Mutating HashSet element loses it

Set<List<Integer>> set = new HashSet<>();

List<Integer> lst = new ArrayList<>(List.of(1,2,3));

set.add(lst);
```

```
lst.add(4); // mutate after adding to set!

System.out.println(set.contains(lst)); // false element is lost!

// FIX: never use mutable objects as Set/Map keys

// Use List.copyOf(lst) as key, or use an immutable record/value object
```

B2: Fix mutable HashSet element mutation bug

```
// B3 TreeSet Comparator trap: comparator returns 0 for 'different' objects

TreeSet<String> set = new TreeSet<>(String.CASE_INSENSITIVE_ORDER);

set.add("hello");

set.add("HELLO"); // comparator says 0 treated as duplicate!

System.out.println(set.size()); // 1 may surprise if you expected 2

// If you need case-insensitive lookup but case-sensitive storage:

// use a HashMap<String,String> with lowerCase key -> original value
```

B3: Fix TreeSet case-insensitive comparator duplicate trap

Part C MCQ: Predict the Output

#	Code	Answer	Why
M C1	Set<String> s=new HashSet<>(); s.add("a"); s.add("a"); println(s.size());	1	Duplicate rejected; add() returns false
M C2	new TreeSet<>().add(null);	NPE	TreeSet rejects null (compareTo(null) throws NPE)
M C3	EnumSet.of(Day.SAT,Day.SUN).contains(null)	NPE	EnumSet rejects null -- NullPointerException
M C4	Set<String> s=new LinkedHashSet<>(List.of("c","a","b")); println(s.iterator().next());	"c"	LinkedHashSet preserves insertion order; first = "c"
M C5	Set<Integer> s=new TreeSet<>(List.of(5,3,1,4,2)); println(s.first());	1	TreeSet is sorted; first() = minimum = 1
M C6	Set.of("a","a")	IllegalArgumentException	Set.of() rejects duplicate elements

REF Quick Reference Cheat Sheet

Topic	Key Points to Remember
Set contract	No duplicates (by equals()). At most one null (most impls). add() returns false not exception.
HashSet internals	Backed by HashMap. O(1) avg ops. Bucket = hash&(cap-1). Treeify at 8 collisions.
HashSet null	One null allowed. Special-cased in bucket 0. TreeSet/EnumSet/CSLS reject null.
LinkedHashSet	HashSet + doubly-linked list for insertion order. O(1) ops. Slightly more memory.
Deduplication + order	new ArrayList<>(new LinkedHashSet<>(list)) -- preserves first-occurrence order.
TreeSet ordering	Uses compareTo/Comparator NOT equals. Inconsistency causes ghost duplicates.
TreeSet null	NO -- compareTo(null) throws NPE. Use a sentinel or Optional wrapper.
TreeSet range ops	subSet/headSet/tailSet: O(log n) live views. first/last/floor/ceiling: O(log n).
EnumSet	Bit vector. O(1) all ops (1 CPU instruction). allOf/noneOf/of/range/complementOf.
EnumSet implementations	RegularEnumSet (<=64 constants, 1 long). JumboEnumSet (>64, long[]).
Set algebra	Union=addAll. Intersection=retainAll. Difference=removeAll. Always copy first!
equals+hashCode	Both MUST be overridden together. Same fields in both. Mutable objects = bugs.
Thread-safe sets	synchronizedSet (mutex, slow). ConcurrentHashMap.newKeySet (concurrent). CSLS (sorted+concurrent).
ConcurrentSkipListSet	Sorted, lock-free, O(log n). Weakly consistent iteration. No null.
Set.copyOf()	Java 10+. Immutable, null-hostile. NOT a HashSet. Use for defensive copies.
Set.of() gotchas	Null -> NPE. Duplicate -> IllegalArgumentException. Order not guaranteed.